- 13 安时积分与 SOC 递推
  - 13.1 运行场景
  - 13.2 决策困难与算法需求
  - 13.3 递推公式
  - 13.4 手算示例
  - 13.5 核心实现
  - 13.6 开环边界
  - 13.7 业务决策
  - 13.8 自测

13 安时积分与 SOC 递推

一句话：安时积分是最基础的 SOC 估计：把电流对时间积分，再除以有效容量。它简单、必要，但只是开环记账。

13.1 运行场景

09:00，夜间充电结束，BMS 需要在全天运行中持续更新 SOC。白天的短时支撑、待命和晚峰放电会不断改变电流方向和持续时间；17:30 的 SOE/SOP 计算依赖这条连续 SOC 轨迹。

BMS 最直接的做法是从一个已知起点开始，对每一拍电流积分：放电就扣除电荷，充电就按库仑效率补回电荷。这个方法计算量小、可在每个采样周期执行，因此是状态估计的基础。

13.2 决策困难与算法需求

安时积分能持续追踪"已流过多少电荷"，但前提是起点、有效容量和电流测量可靠。它本身不能判断 09:00 的起点是否正确，也不能发现传感器零漂是否已经影响 17:30 的晚峰能力。

因此本章先建立可审计的开环 SOC 基线：使用真实 `dt`、当前 `C_eff` 和充电效率推进状态。第 14 章将量化其误差如何累积，第 17 章才引入电压反馈修正。

13.3 递推公式

```
SOC[k+1] = SOC[k] − g(I[k])·dt[k] / (3600·C_eff)
g(I) = I          （放电，I>0）
g(I) = η_c·I      （充电，I<0）
```

符号	含义	单位
`I[k]`	区间平均电流，放电为正	A
`dt[k]`	实际时间间隔	s
`C_eff`	有效容量	Ah

符号	含义	单位
η_c	充电库仑效率	—

容量用 C_{eff}，时间用实际 dt，不能分别替换为铭牌容量和固定 1 s。

13.4 手算示例

统一单体 $C_{eff}=314\ Ah$、$SOC[0]=0.3000$、$dt=60\ s$。

第 1 步：100 A 放电 60 s。

$$
\begin{aligned}
SOC[1] &= 0.3000 - 100\times60/(3600\times314) \\
&= 0.294692
\end{aligned}
$$

SOC 从 30.000% 降到 29.469%。

第 2 步：200 A 放电 60 s。

$$
\begin{aligned}
SOC[2] &= 0.294692 - 200\times60/(3600\times314) \\
&= 0.284076
\end{aligned}
$$

SOC 从 29.469% 降到 28.408%。两分钟共下降约 1.59 个百分点。

13.5 核心实现

下面是完整的单步 SOC 递推函数，支持放电、充电与库仑效率。

```python
def soc_step(soc: float, current_a: float, dt_s: float,
             effective_capacity_ah: float,
             charge_efficiency: float = 0.995) -> float:
    """放电电流为正，充电电流为负，返回下一拍 SOC。"""
    if not 0.0 <= soc <= 1.0:
        raise ValueError("SOC 必须在 0-1")
    if dt_s <= 0 or effective_capacity_ah <= 0:
        raise ValueError("时间间隔和有效容量必须为正")
    if not 0.0 < charge_efficiency <= 1.0:
        raise ValueError("库仑效率必须在 0-1")

    weighted_current = current_a if current_a >= 0 else current_a *
charge_efficiency
    next_soc = soc - weighted_current * dt_s / (3600.0 * effective_capacity_ah)
    return max(0.0, min(1.0, next_soc))


if __name__ == "__main__":
    soc = 0.3000
    soc = soc_step(soc, 100.0, 60.0, 314.0)
    soc = soc_step(soc, 200.0, 60.0, 314.0)
```

```
print(f"两拍后 SOC: {soc:.6f} ({soc:.3%})")
assert round(soc, 6) == 0.284076

charged = soc_step(0.40, -314.0, 360.0, 314.0)
print(f"充电 6 min 后 SOC: {charged:.4f}")
assert round(charged, 4) == 0.4995
```

完整 EKF 对照实验见 `lab/ch05_production_ekf.py`。

13.6 开环边界

安时积分只做记账，不含量测反馈：起点 SOC 错了，误差会保留；电流传感器有零漂，误差会持续累积；它不看电压，无法自我纠正。这正是第 14 章和第 17 章的主题。

13.7 业务决策

全天 SOC 轨迹仍以安时积分作为基础记录，但 17:30 不能将它直接作为晚峰能力依据。EMS 将开环轨迹交给后续章节检查初值、零漂、电压反馈和诊断状态，再决定 SOC 是否可用于 SOE/SOP 计算。

13.8 自测

1. 为什么容量必须使用 `C_eff` 而不是铭牌 314 Ah?
2. 充电时为什么要乘库仑效率 `η_c`?
3. 丢包导致相邻样本间隔 2 s，仍按 1 s 积分会怎样?

▶ 参考答案

1. 电池衰减后铭牌容量偏大，用它作分母会系统性高估 SOC。
2. 外部送入的电荷只有一部分转为可恢复电荷，忽略效率会高估充入量。
3. 会少积分一次，SOC 偏差会累积；必须使用实际 `dt[k]`。